



CITS5017 Deep Learning

Semester 2, 2026

Lab Sheet 3

(Not assessed)

1. Outline

This labsheet is a good exercise to practise what you learned from the lecture notes for Topic 2 (Chapter 11 “*Training Deep Neural Networks*” of the textbook). In each of the two tasks below, we will learn how to train and evaluate some simple MLPs. Task 1 is a classification problem. Task 2 is a regression problem.

2. Tasks

2.1. Task 1

The UCI Machine Learning Repository contains many useful datasets for testing your machine learning algorithm. For this task, we will look at the *wine* dataset. The zip file of this dataset can be downloaded from

<https://archive.ics.uci.edu/static/public/109/wine.zip>

Once unzipping the file, you should see the files *wine.data* and *wine.names*. Although the first file has the extension *.data*, it is actually a *.csv* file and so it can be read using the `read_csv` function from the `pandas` library. Ensure that you use appropriate arguments for the `read_csv` function to read the data from the file correctly. There are 13 attributes for each data instance. The first column is the class ID, labelled as 1, 2, or 3. So there are 14 numbers on each row of the file. You can find the names of the 13 attributes from *wine.names* and put an appropriate column heading for each attribute.

Study the data carefully using various functions from `pandas`, e.g., `head()`, `value_counts()`, `scatter`, etc.

Notice that the 13 attributes have different magnitudes; in particular, the last attribute has values that are significantly larger than the other 12 attributes. Some data normalization would be required. Also, you should modify the class IDs to 0, 1, and 2; otherwise, your code would not work correctly.

Split the dataset using the split ratio 80/20 to form a training set and a validation set.

Subtasks:

- Create an MLP model (`model1`) with 3 hidden layers, each of which should have 5 neurons and should use the GELU activation function. Train and evaluate the model for 500 epochs with *early stopping*. Use one of the more advanced optimizers, such as *Adam*, *Nadam*, etc. To minimize the lengthy outputs from the training epochs, set the argument `verbose` to 0 and, upon the completion of training, illustrate the learning curves. For this dataset, you should be able to get a validation accuracy above 90%. Use an appropriate function from the *scikit-learn* library to show the confusion matrices for your training set and validation set.

- Repeat subtask 1 above by including one more hidden layer to form a new MLP model (`model2`) with 4 hidden layers. You should be able to improve both the training and validation accuracy a little bit more.
- (Optional) Although the class imbalance issue is not so severe for this dataset, you could try incorporating the argument `class_weight` to the `fit()` method.

Modify the `model1` to form a new model, `model3`. The value for `class_weight` must be a dictionary where the *keys* should be the class IDs and the *values* should be the weights for the keys. For instance, `class_weight = {0: w0, 1: w1, 2: w2}`, where `w0`, `w1`, and `w2` should, respectively, be the weight for class 0, class 1, and class 2.

You should see a slight improvement of `model3` over `model1`.

Pitfall: unlike Scikit-Learn classifiers, TensorFlow classifier models output class probabilities rather than class IDs. You will need to use `np.argmax` to convert them to class IDs.

2.2. Task 2

Another wine dataset from the UCI Machine Learning Repository website is the *wine quality* dataset, which can be downloaded from the “DOWNLOAD” button on the link below:

<https://archive.ics.uci.edu/dataset/186/wine+quality>

For this task, we will focus on the *white wine* dataset as it has more data instances (i.e., we will only use the file *winequality-white.csv*). Similar to task 1, this data file can be read using the `read_csv` function from the `pandas` library. However, notice that the character that separates the columns is “;”, not “,”. There are 12 numbers on each row of the file, but only the first 11 numbers are the features (see the file `winequality.names` for more information). The last column, *quality*, contains the scores from 3 to 9 that we want our MLPs to learn to predict. As the target column is numerical and ordered (0 means *bad* wine and 10 means *excellent* wine), it is more meaningful to treat this dataset as a regression problem.

Similar to task 1, study and visualise the data. Split the dataset using the split ratio 80/20 to form a training set and a validation set.

Subtasks:

- Design an MLP with 8 hidden layers each having 5 neurons and using the ReLU activation function. Use “mse” (*mean squared error* (MSE)) as the loss function and set the metric to “mae” (*mean absolute error* (MAE)). This would allow you to get both MSE and MAE when you evaluate your model. Train your model for 500 epochs with early stopping.
- Modify the above MLP to form a new model as follows: let the first 4 hidden layers use ReLU; let the last 4 hidden layers and the output layer use `tanh`. After these 9 layers, append a `Rescaling` layer to scale and offset the output values to the range $3 \cdots 9$. You should find this model perform better than the previous model.
- (Optional) You could modify all the hidden layers to use `tanh`. Most of the time, a further (but small) improvement can be achieved.